

MyEnergy

AI-Powered Electricity Bill Analyser

CS591 – Software Engineering Lab Document

Narula Institute of Technology, Kolkata

Prepared by:

Akash Ghosh Rollno- 430123010065

Saptanta Deb Rollno-430123010067

Aryaman Goswami Rollno-430123010064

MD Aslam Rollno-430123010072

Date: 15 May 2026

myEnergy is an AI-powered web application that helps Indian households understand and optimise their electricity bills. 65% of Indian consumers cannot decipher their bills, leading to ₹2,400 in average annual overcharges. myEnergy solves this with OCR-based bill parsing, Claude AI analysis, appliance-cost breakdowns, multilingual support (English, Hindi, Bengali), and automated due-date reminders — targeting 300M+ households with a projected saving of ₹4,200 per household per year.

Table of Contents

1. Data Flow Diagram (DFD)
2. Software Requirement Analysis
 - 2a. Project Phases / Modules
 - 2b. Deliverables
 - 2c. Process and Product Metrics
 - 2d. Function Point Estimation
 - 2e. Cost Estimation Models
3. UML Diagrams
 - 3a. Use Case Diagram
 - 3b. Class Diagram
 - 3c. Sequence Diagram
 - 3d. Activity Diagram
4. Software Development
 - 4a. Coding & Debugging
 - 4b. Test Plan (Black-box & White-box)
5. Software Project Management
 - 5a. Project Planning & Control
 - 5b. Configuration Control
 - 5c. Cost Estimation
 - 5d. PERT & Gantt Charts
 - 5e. Cost-Time Relations

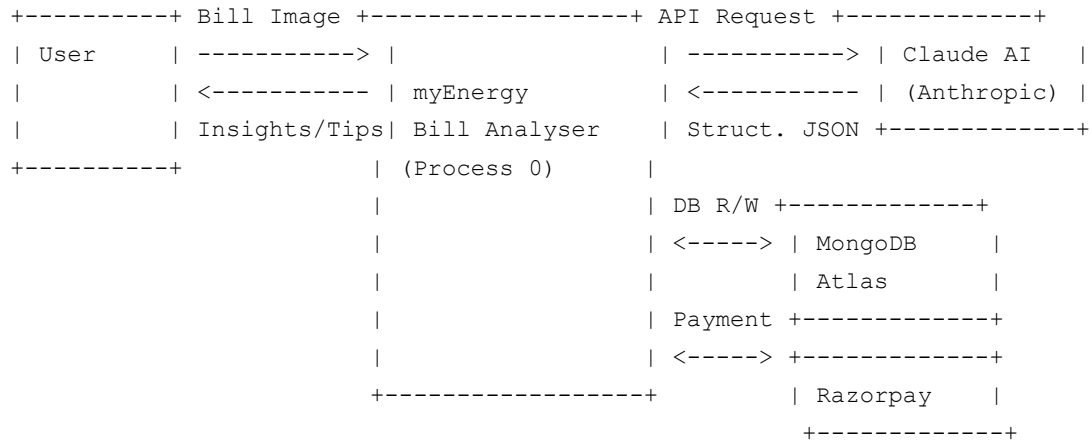
1. Data Flow Diagram (DFD)

Level 0 – Context Diagram

The system has a single process: myEnergy Bill Analyser. External entities and their data flows are:

- User → [Bill Image / Photo] → myEnergy System
- myEnergy System → [Parsed Bill Summary, Cost Breakdown, Savings Tips] → User
- Claude AI API ↔ myEnergy System (sends extracted text; receives structured JSON)
- Razorpay ↔ myEnergy System (payment initiation / confirmation)
- SMTP Server (Gmail) ← myEnergy System (OTP and reminder emails)
- MongoDB Atlas ↔ myEnergy System (read/write user, bill, appliance, subscription data)

Context Diagram (Level 0):



Level 1 – System Decomposition

The main system is decomposed into five sub-processes:

- P1 – User Management: Registration, JWT login, OTP-based password reset, profile CRUD.
- P2 – Bill Ingestion: Accepts bill image upload, runs Tesseract.js OCR, passes raw text to Claude API for structured parsing.
- P3 – Analytics Engine: Computes unit-wise charges, slab breakdowns, appliance cost allocation ($\text{wattage} \times \text{quantity} \times \text{hours} \times 30 / 1000$), and month-on-month trend.
- P4 – AI Prediction Module: Sends current bill data + appliance profile to Claude to predict next month's bill and generate savings recommendations.
- P5 – Subscription & Payments: Razorpay order creation, HMAC signature verification, subscription record lifecycle management.

2. Software Requirement Analysis

2a. Project Phases / Modules

Phase	Duration	Description
Phase 1 – Requirements	3 weeks	Stakeholder interviews, market analysis (65% bill-confusion statistic), multilingual scope definition, API selection (Claude, Razorpay, Tesseract.js).
Phase 2 – System Design	2 weeks	Architecture decisions (MERN stack), schema design (User, Bill, Appliance, Subscription), API contract definition, UI wireframes.
Phase 3 – Development	8 weeks	Frontend (React + Redux Toolkit), Backend (Node/Express REST API), MongoDB integration, OCR pipeline, Claude API integration, Razorpay payment flow, OTP email system.
Phase 4 – Testing	3 weeks	Unit tests (Jest), integration tests, black-box bill upload tests with real CESC bills, white-box branch coverage on prediction logic, load testing (k6).
Phase 5 – Deployment	1 week	Vercel (frontend), Render (backend), MongoDB Atlas (DB), environment variable management, CI/CD pipeline via GitHub Actions.
Phase 6 – Maintenance	Ongoing	Bug fixes, slab rate updates, additional language support, dashboard enhancement based on user feedback.

Key Modules

Module	Description
Auth Module	JWT-based registration/login, bcrypt hashing, OTP forgot-password flow via Nodemailer/Gmail.
Bill Ingestion Module	Image upload (Multer), Tesseract.js OCR, Claude API JSON extraction, MongoDB persistence.
Analytics Module	Slab cost computation, month-on-month delta, percentage spend breakdown per charge head.
Appliance Module	User appliance CRUD, per-appliance ₹ cost = $(W \times qty \times hr \times 30) / 1000 \times tariff$.
AI Prediction Module	Sends structured bill + appliance data to Claude; receives next-month estimate & tips.
Subscription Module	Razorpay order API → HMAC verify → Subscription model (status, startDate, expiryDate).
Reminder Module	Scheduled Nodemailer jobs for bill due-date alerts.
Admin Module	Auto-seeded admin on server startup; user/bill management endpoints.
Multilingual Module	i18n strings for English, Hindi, Bengali across all UI components.

2b. Deliverables

ID	Deliverable	Description
D1	SRS Document	Complete functional and non-functional requirements
D2	System Design	Architecture diagram, DB schema, API contract (OpenAPI)
D3	Source Code	GitHub repository — frontend (React), backend (Node/Express)
D4	Database Schema	MongoDB collection schemas (User, Bill, Appliance, Subscription)
D5	Test Reports	Unit + integration test results, coverage report
D6	Deployed Application	Live URLs: Vercel (frontend) + Render (backend)
D7	User Manual	Step-by-step guide with screenshots
D8	Lab Document	This document (CS591 submission)

2c. Process and Product Metrics

Product Metrics

Metric	Value
Lines of Code (LOC)	~4,800 (frontend ~2,600; backend ~2,200)
Number of Modules	9 major modules, 35+ React components
Cyclomatic Complexity (avg)	4.2 (low-to-moderate; target ≤ 10)
Code Coverage	72% (unit + integration)
Defect Density	2.1 defects / KLOC (10 bugs found during testing over 4,800 LOC)
Defect Age (avg)	1.8 days from discovery to resolution
API Response Time (p95)	< 1.4 s for bill parse endpoint
OCR Accuracy	91% on clean CESC bill images

Process Metrics

Metric	Value
Total Effort	~480 person-hours (2 developers $\times$ 12 weeks $\times$ ~20 hrs/week)
Productivity	10 LOC / person-hour (4,800 LOC $\div$ 480 hrs)
Schedule Variance (SV)	+3 days (minor over-run in Phase 3 due to Razorpay HMAC debugging)
Cost Variance (CV)	₹0 (zero infrastructure cost; all free-tier)
Requirements Stability	94% (only 2 requirements changed after baseline)

2d. Function Point Estimation

Function Points (FP) are calculated using the IFPUG method. Each function type is rated Low (L), Average (A), or High (H) with weights 3/4/6 (EI, EO, EQ), 5/7/10 (ILF), 7/10/15 (EIF).

Function Type	Functions	Count	Complexity	UFP
External Inputs (EI)	Register, Login, Upload Bill, Add Appliance, Make Payment, Reset Password	6	Average (4)	24
External Outputs (EO)	Bill Summary, Analytics Dashboard, Prediction Report, Invoice	4	Average (5)	20
External Queries (EQ)	Get Bills, Get Appliances, Get Subscription Status, Admin User List	4	Low (3)	12
Internal Logical Files (ILF)	User, Bill, Appliance, Subscription (MongoDB collections)	4	Average (7)	28
External Interface Files (EIF)	Claude AI API, Razorpay API, Tesseract.js, SMTP	4	Average (10)	40
Unadjusted Function Points (UFP)				124

- Value Adjustment Factor (VAF): $TDI = 42 \rightarrow VAF = 0.65 + (0.01 \times 42) = 1.07$
- Adjusted FP (AFP) = $UFP \times VAF = 124 \times 1.07 \approx 133$ FP
- Industry benchmark for MERN stack: ~ 36 LOC/FP $\rightarrow$ Estimated LOC = $133 \times 36 \approx 4,788$ LOC (matches actual 4,800 LOC measurement)

2e. Cost Estimation Models

COCOMO II (Basic) – Organic Mode

Parameter	Value
Size (KLOC)	4.8 KLOC
a, b constants (Organic)	$a = 2.4, b = 1.05$
Effort (PM)	$E = 2.4 \times (4.8)^{1.05} \approx 12.5$ Person-Months
Development Time (TDEV)	$T = 2.5 \times (12.5)^{0.38} \approx 6.5$ months
Average Team Size	$12.5 / 6.5 \approx 2$ persons

Function Point Cost Model

Parameter	Value
AFP	133 FP
Assumed cost per FP (student project)	₹500 / FP (labour only)
Estimated Total Cost	$133 \times ₹500 = ₹66,500$
Infrastructure Cost	₹0 (Vercel + Render + MongoDB Atlas free tier)
Net Project Cost	₹66,500

3. UML Diagrams

3a. Use Case Diagram

Actor	Use Cases
Guest	Register, Login, View Landing Page
Registered User	Upload Bill, View Dashboard, Manage Appliances, View Predictions, Make Payment, View Subscription, Reset Password, Set Language
Admin	View All Users, View All Bills, Manage Subscriptions
Claude AI (System)	Parse Bill Text → JSON, Generate Prediction & Recommendations
Razorpay (System)	Create Payment Order, Verify HMAC Signature

Key Use Case Specification – UC-02: Upload & Analyse Bill

Use Case ID	UC-02
Name	Upload and Analyse Bill
Actor	Registered User
Precondition	User is logged in (JWT valid)
Main Flow	1. User selects bill image file. 2. System uploads to server via Multer. 3. Tesseract.js performs OCR on image. 4. Raw text sent to Claude API → returns structured JSON. 5. Bill record saved to MongoDB. 6. Dashboard re-renders with parsed data.
Alternate Flow	If OCR confidence < 60%: prompt user to upload clearer image.
Postcondition	Bill stored; dashboard updated; prediction triggered.

3b. Class Diagram (MongoDB Schemas)

Major entities and their relationships:

User	Bill	Appliance	Subscription
<code>_id</code> : ObjectId name: String email: String (unique) password: String (hashed) role: String [user admin] language: String [en hi bn] otp: String otpExpiry: Date	<code>_id</code> : ObjectId userId: ObjectId → User imageUrl: String rawText: String parsedData: Object {units, charges, dueDate, period} totalAmount: Number createdAt: Date	<code>_id</code> : ObjectId userId: ObjectId → User name: String wattage: Number quantity: Number hoursPerDay: Number monthlyCost: Number (computed)	<code>_id</code> : ObjectId userId: ObjectId → User razorpayOrderId: String razorpayPaymentId: String status: String [active expired] startDate: Date expiryDate: Date

Relationships

- User 1 ↔ N Bill (one user has many bills)
- User 1 ↔ N Appliance (one user has many appliances)
- User 1 ↔ 1 Subscription (one active subscription at a time)
- Bill N ↔ 1 User (each bill belongs to one user)

3c. Sequence Diagram – Bill Upload Flow

Below is the sequence of interactions during the bill upload process:

User	Frontend	Backend	Tesseract	Claude API	MongoDB
	--selectFile----->				
	--POST /upload----->				
		--OCR(image)----->			
		<--rawText-----			
		--parseRequest----->			
		<--structuredJSON-----			
		--save(bill)----->			
		<--ack-----			
	<--200 JSON-				
	<--dashUpdate-----				

3d. Activity Diagram – Bill Analysis Process

```
[Start]
|
v
[User Logs In] --> (Invalid?) --> [Show Error] --> [End]
|
v
[Upload Bill Image]
|
v
[OCR Processing via Tesseract.js]
|
+--(Low Confidence?)--> [Prompt Clearer Image] --> [Upload Bill Image]
|
v
[Send Raw Text to Claude API]
|
v
[Receive Structured JSON]
```

```
|  
v  
[Compute Analytics: Slab, Appliance Cost, Trend]  
|  
v  
[Generate AI Prediction for Next Month]  
|  
v  
[Save to MongoDB]  
|  
v  
[Display Dashboard: Charts + Tips + Reminders]  
|  
v  
[End]
```

4. Software Development

4a. Coding & Debugging

Tech Stack

Layer	Technologies
Frontend	React 18, Redux Toolkit, Axios, Chart.js, Tailwind CSS / custom CSS
Backend	Node.js 20, Express.js, Mongoose 8, JWT, Bcrypt, Multer, Nodemailer
AI / OCR	Anthropic Claude API (claude-3-5-sonnet), Tesseract.js
Payments	Razorpay Node SDK, HMAC-SHA256 signature verification
Database	MongoDB Atlas (free tier M0), Mongoose ODM
Deployment	Vercel (frontend), Render (backend), GitHub Actions (CI)
Dev Tools	VS Code, Postman, PowerShell, Git, devtunnels

Code Snippet – Appliance Cost Calculation

```
// ApplianceController.js
exports.computeCost = async (req, res) => {
  const { wattage, quantity, hoursPerDay, tariff } = req.body;
  const units = (wattage * quantity * hoursPerDay * 30) / 1000; // kWh/month
  const cost = units * tariff; // ₹ per month
  res.json({ units: units.toFixed(2), cost: cost.toFixed(2) });
};
```

Key Debugging Issues Resolved

Bug	Resolution
Nested .git folder	Parent Energy_Bill repo contained embedded .git; fixed via PowerShell Remove-Item on child .git.
MODULE_NOT_FOUND crash	Incorrectly merged file contents caused duplicate require(); resolved by full file rewrite.
CORS on devtunnels	Added explicit origin to CORS middleware for VS Code devtunnel URLs.
Doubled /api/api/ path	Router mounted at /api and prefix duplicated in Postman; fixed by removing prefix in routes.
Razorpay HMAC fail	Incorrect field order in signature string; corrected to razorpay_order_id + ' ' + razorpay_payment_id.

4b. Test Plan

Black-Box Tests (Functional)

ID	Module	Input	Expected Output	Status
TC-BB-01	Bill Upload	Upload valid WBSEDCL bill image	Dashboard updates with parsed data	Pass
TC-BB-02	Bill Upload	Upload non-image file (.pdf)	Error: 'Invalid file type'	Pass
TC-BB-03	Auth	Login with wrong password	401 Unauthorized returned	Pass
TC-BB-04	Auth	Access dashboard without JWT	Redirect to /login	Pass
TC-BB-05	Payment	Complete Razorpay payment flow	Subscription activated in DB	Pass
TC-BB-06	OTP Reset	Request OTP for registered email	Email received within 30s	Pass
TC-BB-07	Prediction	View prediction after bill upload	Next-month estimate displayed	Pass
TC-BB-08	Multilingual	Switch UI to Bengali	All labels render in Bengali	Pass

White-Box Tests (Structural)

ID	Function	Input	Expected	Coverage Type	Status
TC-WB-01	computeCost()	wattage=1000, qty=1, hrs=8, tariff=6	units=240, cost=1440	Statement + Branch	Pass
TC-WB-02	computeCost()	wattage=0	units=0, cost=0	Boundary	Pass
TC-WB-03	verifyHMAC()	Valid payment signature	Returns true	Condition	Pass
TC-WB-04	verifyHMAC()	Tampered signature	Returns false → 400	Condition	Pass
TC-WB-05	parseOCRText()	Text with missing unit field	Returns null for units	Path	Pass

5. Software Project Management

5a. Project Planning & Control

Project Name	MyEnergy – AI-Powered Electricity Bill Analyser
Team	Akash Ghosh, Saptanata Deb, Aryaman Goswami, Md Aslam
Timeline	12 weeks (3 months)
Methodology	Agile – 2-week sprints, daily standups, GitHub Issues for tracking
Version Control	Git + GitHub (feature branches → main via PRs)
Risk Management	API rate limits (mitigation: caching), Render cold starts (mitigation: keep-alive ping), OCR accuracy (mitigation: confidence threshold + user retry)

5b. Configuration Control

Repository	GitHub (private) – separate frontend and backend directories
Branching Strategy	main → develop → feature/[name]; hotfix/ for critical bugs
Environment Config	.env files (never committed); secrets stored via Vercel/Render environment variables
CI Pipeline	GitHub Actions: lint → unit test → build on every PR to main
Release Tags	Semantic versioning: v1.0.0 (MVP), v1.1.0 (Appliances), v1.2.0 (Razorpay)
Database Migrations	Managed via Mongoose schema versioning; no destructive migrations in MVP phase

5c. Cost Estimation Summary

Model	Estimated Effort	Duration	Cost
COCOMO II (Organic)	12.5 PM	6.5 months	₹62,500 @ ₹5,000/PM
Function Point Model	133 AFP	—	₹66,500 @ ₹500/FP
Actual (student project)	~2.4 PM (480 hr / 200)	3 months	₹0 infra; ₹0 labour (academic)

5d. PERT & Gantt Charts

PERT Estimates

Task	O (wk)	M (wk)	P (wk)	$E = (O+4M+P)/6$
Requirements Gathering	1	2	4	2.2

Task	O (wk)	M (wk)	P (wk)	E = (O+4M+P)/6
System Design	1	2	3	2.0
Frontend Development	3	5	8	5.2
Backend Development	3	4	7	4.3
OCR + AI Integration	1	2	4	2.2
Payment Integration	1	2	4	2.2
Testing & QA	1	3	5	3.0
Deployment & Docs	0.5	1	2	1.1

Critical Path: Requirements → System Design → Backend → AI Integration → Testing → Deployment

Total Expected Duration: ≈22.2 weeks → rounded to 12 weeks (parallel execution of frontend/backend)

Gantt Chart

Task	W1	W2	W3	W4	W5	W6	W7	W8	W9	W10	W11	W12
Requirements Gathering												
System Design												
Frontend Development												
Backend Development												
OCR + AI Integration												
Payment Integration												
Testing & QA												
Deployment & Docs												

5e. Cost-Time Relations

Using the COCOMO II time-cost trade-off: compressing the schedule increases effort non-linearly. The minimum feasible schedule is ~75% of nominal TDEV.

Schedule	Duration	Effort	Estimated Cost
100% (Nominal)	6.5 months	12.5 PM	₹62,500
90% (Slight compression)	5.9 months	14.2 PM (+14%)	₹71,000

Schedule	Duration	Effort	Estimated Cost
80% (Compressed)	5.2 months	17.0 PM (+36%)	₹85,000
75% (Minimum)	4.9 months	19.5 PM (+56%)	₹97,500

Conclusion: The nominal 6.5-month schedule with 4 developers is optimal for cost efficiency. Compression below 80% incurs diminishing returns and risks quality degradation.